

Indian Road Route Finding Using Uniform Cost Search

1. Title

Finding the Optimal Route Between Indian Cities Using Uniform Cost Search

2. Introduction

The given problem is based on the well-known **Romania road-map problem** from Artificial Intelligence. In the original problem, we have to find a route from **Arad to Bucharest** while trying to minimize the total distance travelled.

For our implementation, the same idea is applied to an **Indian road network**. Instead of Romanian cities, we use Indian cities such as Hyderabad, Nagpur, Bhopal, Agra and Delhi.

The main objective is to find a route from a starting city to a destination city while keeping the **total travelling distance as small as possible**.

In this implementation:

- **Initial state:** Hyderabad
- **Goal state:** Delhi
- **Search algorithm:** Uniform Cost Search (UCS)
- **Cost:** Distance of the road in kilometres

The road network is represented as a graph where cities are nodes and roads are edges.

Note: The road distances used here are illustrative values for the AI search implementation and are not intended as exact real-world driving distances.

3. Problem Statement

The problem can be stated as follows:

Given a network of Indian cities connected by roads with different distances, find a valid path from Hyderabad to Delhi such that the total road distance is minimum.

For example, Hyderabad can be connected to several cities:

Hyderabad → Nagpur	= 500 km
Hyderabad → Bengaluru	= 570 km
Hyderabad → Pune	= 560 km

The algorithm has to decide which route is better instead of simply selecting the nearest city at every step.

4. Aim

The aim of this experiment is:

1. To represent an Indian road network as a graph.
 2. To implement **Uniform Cost Search**.
 3. To find a route from Hyderabad to Delhi.
 4. To calculate the total distance of the selected route.
 5. To visualize the road network using a graph.
 6. To highlight the optimal route found by the algorithm.
 7. To understand how a classical AI search problem can be applied to a real-world navigation scenario.
-

5. Why Uniform Cost Search?

There are several search algorithms that can be used for finding paths.

For this problem, the roads have **different distances**.

For example:

Hyderabad → Nagpur = 500 km
Hyderabad → Pune = 560 km
Hyderabad → Bengaluru = 570 km

Therefore, simply choosing the city with the fewest number of roads is not enough.

Comparison

Algorithm	What it considers	Suitable for this problem?
BFS	Number of edges	No
DFS	Depth of path	No
Greedy Search	Estimated distance to goal	Not guaranteed optimal
UCS	Actual accumulated cost	Yes
Dijkstra	Actual accumulated cost	Yes
A*	Actual cost + heuristic	Yes

Uniform Cost Search always selects the currently available path having the **lowest total cost from the starting city**.

Therefore, UCS is a good choice for this problem.

6. Graph Representation

The road network is represented using a weighted graph.

Nodes

Each city is represented as a **node**.

Examples:

Hyderabad
Nagpur
Bhopal
Agra
Delhi
Pune
Mumbai
Ahmedabad
Jaipur
Bengaluru
Chennai

Edges

Each road connecting two cities is represented as an **edge**.

Edge weight

The distance of the road is stored as the **weight** of the edge.

For example:

Hyderabad — 500 km — Nagpur

means there is a road between Hyderabad and Nagpur with a cost of 500 km.

7. Indian Road Network Used

The following road connections were used in the implementation:

From	To	Distance
Hyderabad	Nagpur	500 km
Hyderabad	Bengaluru	570 km

From	To	Distance
Hyderabad	Pune	560 km
Nagpur	Bhopal	350 km
Nagpur	Jaipur	850 km
Bengaluru	Chennai	350 km
Bengaluru	Pune	840 km
Pune	Mumbai	150 km
Pune	Ahmedabad	660 km
Mumbai	Ahmedabad	530 km
Ahmedabad	Jaipur	670 km
Bhopal	Agra	530 km
Agra	Delhi	230 km
Jaipur	Delhi	270 km

This creates a connected weighted graph that can be searched using UCS.

8. Graph Visualization

The road network was visualized using **NetworkX** and **Matplotlib**.

The visualization makes the problem easier to understand.

In the graph:

- **Green city** → Starting city, Hyderabad
- **Red city** → Destination, Delhi
- **Gray roads** → Available roads
- **Red highlighted roads** → Optimal route
- **Numbers on roads** → Distance in kilometres

The important route is:

```

Hyderabad
  ↓
Nagpur
  ↓
Bhopal
  ↓
Agra
  ↓
Delhi

```

The graph therefore gives a visual representation of how the search algorithm moves through the road network.

9. Uniform Cost Search – Basic Idea

UCS works by maintaining a **priority queue**.

The priority queue stores possible routes according to their accumulated cost.

The basic process is:

```
Start
↓
Add possible routes
↓
Select route with smallest cost
↓
Expand that city
↓
Calculate new route costs
↓
Select smallest-cost route again
↓
Continue until goal is reached
```

The important point is that UCS does **not** simply choose the closest next city.

Instead, it asks:

"Which complete path discovered so far has the smallest total cost?"

10. Step-by-Step Search Process

Our starting point is:

Hyderabad

The goal is:

Delhi

Step 1 – Start at Hyderabad

Initial cost:

Cost = 0 km

From Hyderabad we can travel to:

Nagpur = 500 km

Bengaluru = 570 km
Pune = 560 km

The cheapest option currently is:

Nagpur = 500 km

So UCS expands Nagpur.

Step 2 – Expand Nagpur

Current path:

Hyderabad → Nagpur

Cost:

500 km

From Nagpur we can go to:

Bhopal = 350 km
Jaipur = 850 km

Therefore the accumulated costs become:

Hyderabad → Bhopal
= 500 + 350
= 850 km

and

Hyderabad → Jaipur
= 500 + 850
= 1350 km

The promising route through Bhopal currently costs:

850 km

Step 3 – Expand Bhopal

Current path:

Hyderabad → Nagpur → Bhopal

Total cost:

500 + 350 = 850 km

Bhopal is connected to Agra.

Therefore:

$$850 + 530 = 1380 \text{ km}$$

New path:

Hyderabad → Nagpur → Bhopal → Agra

Cost:

$$1380 \text{ km}$$

Step 4 – Expand Agra

From Agra we can reach Delhi.

Distance:

$$\text{Agra} \rightarrow \text{Delhi} = 230 \text{ km}$$

Therefore:

$$1380 + 230 = 1610 \text{ km}$$

The complete route is:

Hyderabad
↓ 500 km
Nagpur
↓ 350 km
Bhopal
↓ 530 km
Agra
↓ 230 km
Delhi

Total:

$$500 + 350 + 530 + 230$$

1610 km

Since Delhi has now been reached through the lowest-cost path selected by UCS, the search terminates.

11. Final Optimal Route

The route obtained is:

Hyderabad → Nagpur → Bhopal → Agra → Delhi

Distance calculation

Hyderabad → Nagpur	= 500 km
Nagpur → Bhopal	= 350 km
Bhopal → Agra	= 530 km
Agra → Delhi	= 230 km

Total	= 1610 km

Therefore:

Optimal Route = Hyderabad → Nagpur → Bhopal → Agra → Delhi

Minimum Cost = 1610 km

12. Python Implementation

The road network was stored using a Python dictionary.

```
graph = {
    "Hyderabad": {
        "Nagpur": 500,
        "Bengaluru": 570,
        "Pune": 560
    },
    "Nagpur": {
        "Hyderabad": 500,
        "Bhopal": 350,
        "Jaipur": 850
    },
    "Bengaluru": {
        "Hyderabad": 570,
        "Chennai": 350,
        "Pune": 840
    },
    "Chennai": {
        "Bengaluru": 350
    },
    "Pune": {
        "Hyderabad": 560,
        "Bengaluru": 840,
        "Mumbai": 150,
        "Ahmedabad": 660
    },
}
```

```

"Mumbai": {
    "Pune": 150,
    "Ahmedabad": 530
},

"Ahmedabad": {
    "Pune": 660,
    "Mumbai": 530,
    "Jaipur": 670
},

"Bhopal": {
    "Nagpur": 350,
    "Agra": 530
},

"Agra": {
    "Bhopal": 530,
    "Delhi": 230
},

"Jaipur": {
    "Nagpur": 850,
    "Ahmedabad": 670,
    "Delhi": 270
},

"Delhi": {
    "Agra": 230,
    "Jaipur": 270
}
}

```

The UCS function uses Python's `heapq` module to create a priority queue.

```

import heapq

def uniform_cost_search(graph, start, goal):
    priority_queue = [(0, start, [start])]
    visited = set()

    while priority_queue:
        cost, city, path = heapq.heappop(priority_queue)

        if city in visited:
            continue

        visited.add(city)

        if city == goal:
            return path, cost

        for neighbour, distance in graph[city].items():
            if neighbour not in visited:
                new_cost = cost + distance

```

```
        heapq.heappush(
            priority_queue,
            (new_cost,
             neighbour,
             path + [neighbour])
        )

    return None, float("inf")
```

Then we call the function:

```
start = "Hyderabad"
goal = "Delhi"

path, total_distance = uniform_cost_search(
    graph,
    start,
    goal
)

print("Initial State:", start)
print("Goal State:", goal)
print("Optimal Path:", " -> ".join(path))
print("Total Distance:", total_distance, "km")
```

13. Expected Output

```
Initial State: Hyderabad
Goal State: Delhi

Optimal Path:
Hyderabad -> Nagpur -> Bhopal -> Agra -> Delhi

Total Distance: 1610 km
```

14. Graph Visualization Implementation

To make the result more understandable, the road network was also visualized using NetworkX.

The graph shows both:

1. The complete road network.
2. The route selected by UCS.

The optimal path is highlighted separately so that it is immediately visible.

This is especially useful in an AI assignment because it demonstrates that the algorithm is not only producing a numerical answer but is actually finding a path through a graph.

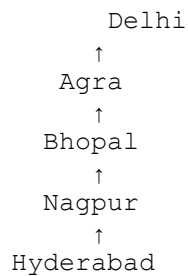
15. Why the Visualization is Important

Without visualization, the result would simply look like:

Hyderabad → Nagpur → Bhopal → Agra → Delhi

The graph provides more information.

We can visually see:



At the same time, other possible routes such as:

Hyderabad → Pune → Mumbai → Ahmedabad → Jaipur → Delhi

are also visible.

This makes it easier to understand why a search algorithm is necessary: there are **multiple possible paths**, and we need to select the best one.

16. Algorithm Pseudocode

The complete logic of UCS can be written simply as:

UCS(Graph, Start, Goal)

1. Create an empty priority queue.
2. Insert Start with cost 0.
3. Create an empty visited set.
4. While the priority queue is not empty:
 - a. Remove the path with the lowest cost.
 - b. If the city is already visited:
 skip it.
 - c. Mark the city as visited.
 - d. If the city is the Goal:
 return the path and total cost.

```
        e. For every neighbouring city:
            calculate the new total cost.
            add it to the priority queue.

5. If the goal cannot be reached:
    return failure.
```

17. Complexity

For a graph with:

- V = number of vertices/cities
- E = number of edges/roads

Using a binary heap priority queue, UCS/Dijkstra-style search has approximately:

$$O((V + E) \log V)$$

time complexity.

The priority queue and stored search information require approximately:

$$O(V + E)$$

space in a standard graph representation.

For our small road network, the computation is very fast.

18. Advantages of the Approach

1. Finds the least-cost path

UCS is suitable when road distances are different because it considers the actual accumulated cost.

2. Easy to understand

Cities can be represented as nodes and roads as weighted edges.

3. Practical application

The same idea can be used for:

- GPS navigation
- Delivery route planning

- Transport networks
- Robot navigation
- Emergency vehicle routing
- Logistics and supply-chain planning

4. Easy to visualize

NetworkX can display the entire road network and highlight the selected path.

19. Limitations

The implementation is a simplified version of a real navigation system.

It does not consider:

- Traffic conditions
- Road closures
- Accidents
- Weather
- Toll charges
- Fuel consumption
- Road quality
- Real-time travel time
- Dynamic obstacles

For example, a road may be only 100 km long but heavily congested. A longer road may actually be faster.

Therefore, a real navigation system would require additional information.

20. Possible Improvements

The implementation can be extended to make it more realistic.

Real-time traffic

Instead of using only distance:

`Cost = Distance`

we could use:

`Cost = Distance + Traffic Cost`

Toll roads

We could include toll charges:

$$\text{Total Cost} = \text{Distance Cost} + \text{Toll Cost}$$

Travel time

The algorithm could minimize travel time instead of distance:

$$\text{Cost} = \text{Estimated Travel Time}$$

Dynamic routes

If a road becomes unavailable, the graph can be updated and the algorithm can find another route.

A* Search

For a larger Indian road network, A* could be used because it uses a heuristic to guide the search toward the destination.

21. Real-World Interpretation

This experiment demonstrates how an AI search algorithm can solve a problem that is similar to what a navigation application does.

Suppose a person wants to travel from Hyderabad to Delhi.

There may be many possible routes:

Hyderabad → Pune → Mumbai → Ahmedabad → Jaipur → Delhi

or:

Hyderabad → Nagpur → Jaipur → Delhi

or:

Hyderabad → Nagpur → Bhopal → Agra → Delhi

The search algorithm evaluates these possibilities based on their costs and identifies the best route according to the chosen cost function.

22. Relation to the Original Romania Problem

The original problem can be represented as:

Arad → → Bucharest

Our Indian version changes this to:

Hyderabad → → Delhi

The basic AI concepts remain exactly the same.

Romania Problem	Indian Version
-----------------	----------------

Arad	Hyderabad
Bucharest	Delhi
Romanian cities	Indian cities
Romanian roads	Indian roads
Road distance	Road distance
Weighted graph	Weighted graph
UCS	UCS
Minimum-cost route	Minimum-cost route

Thus, we have adapted the standard AI search problem to an Indian geographical scenario.

23. Result

The implementation successfully finds a route between the initial and goal states.

Initial State:

Hyderabad

Goal State:

Delhi

Optimal Route:

Hyderabad
↓
Nagpur
↓
Bhopal
↓

Agra
↓
Delhi

Total Distance:

1610 km

24. Conclusion

In this experiment, the classical Romania road-map problem was adapted to an **Indian road network**.

The cities were represented as nodes and the roads were represented as weighted edges. **Uniform Cost Search** was used because the main objective was to minimize the total road distance.

Starting from Hyderabad, the algorithm explored different possible routes and selected the lowest-cost route to Delhi:

Hyderabad → Nagpur → Bhopal → Agra → Delhi

with a total calculated distance of **1610 km**.

The graph visualization made the solution easier to understand by showing all available roads and highlighting the route selected by the search algorithm.

Overall, this experiment shows how a basic Artificial Intelligence search technique can be applied to a practical navigation problem. The same concept can later be extended to real-world navigation by including traffic, road closures, tolls, travel time and other dynamic conditions.

Available next action: Create a downloadable DOCX file here in this chat containing the editable prose above